

WuLab — Implementation Plan: Phase 1 MVP

Siam Master Concrete Co., Ltd.

Updated: 2026-05-03 | **เป้าหมาย:** ใช้งานได้จริงภายใน 5–7 วัน **Stack:** Next.js 14 · TypeScript · Tailwind CSS · Prisma · SQLite · next-intl · Resend

ภาพรวม

- วันที่ 1 → Setup + Database + Navbar/Footer

วันที่ 2 → Landing Page (Hero, Strengths, CTA)

วันที่ 3 → Products (list + detail + API)

วันที่ 4 → RFQ Form + Email + Success page

วันที่ 5 → i18n + About Page + Track page

วันที่ 6 → ทดสอบทุกหน้า + Bug fix

วันที่ 7 → Deploy Vercel Production

ไฟล์ที่จะสร้าง/แก้ไข (File Map)

```
wulab/
├── app/
│   ├── [locale]/
│   │   ├── layout.tsx           ← root layout + font + i18n provider
│   │   ├── page.tsx           ← Landing page
│   │   ├── about/page.tsx
│   │   ├── products/
│   │   │   ├── page.tsx
│   │   │   └── [slug]/page.tsx
│   │   ├── rfq/
│   │   │   ├── page.tsx
│   │   │   ├── success/page.tsx
│   │   │   └── track/page.tsx
│   ├── api/
│   │   ├── products/route.ts
│   │   ├── products/[slug]/route.ts
│   │   ├── rfq/route.ts
│   │   └── rfq/[ref]/route.ts
│   ├── globals.css
│   └── layout.tsx              ← root HTML wrapper
├── components/
│   ├── layout/
│   │   ├── Navbar.tsx
│   │   └── Footer.tsx
│   ├── home/
│   │   └── HeroSection.tsx
```

```
| | | | TrustStrip.tsx
| | | | StrengthsSection.tsx
| | | | UseCaseSection.tsx
| | | | CTABand.tsx
| | | | products/
| | | | | ProductCard.tsx
| | | | | ProductSpec.tsx
| | | | rfq/
| | | | | RFQForm.tsx
| | | | | TrackForm.tsx
| | | | lib/
| | | | | prisma.ts
| | | | | email.ts
| | | | | ref-code.ts
| | | | messages/
| | | | | th.json
| | | | | en.json
| | | | prisma/
| | | | | schema.prisma
| | | | | seed.ts
| | | | middleware.ts
| | | | next.config.ts
| | | | tailwind.config.ts
```

Task 1: Project Setup

เวลา: ~2 ชั่วโมง

ขั้นตอน

1.1 สร้างโปรเจค

```
npx create-next-app@latest wulab \
  --typescript \
  --tailwind \
  --eslint \
  --app \
  --src-dir=false \
  --import-alias="@/*"
```

```
cd wulab
```

1.2 ติดตั้ง dependencies

```
npm install prisma @prisma/client
npm install next-intl
npm install resend
npm install bcryptjs
```

```
npm install jsonwebtoken
npm install @types/bcryptjs @types/jsonwebtoken --save-dev
```

1.3 ตั้งค่า Prisma

```
npx prisma init --datasource-provider sqlite
```

แก้ไข `prisma/schema.prisma`:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model Product {
  id          String   @id @default(cuid())
  slug        String   @unique
  nameTh      String
  nameEn      String
  descTh      String
  descEn      String
  category    String
  imageUrl    String?
  datasheetUrl String?
  specs       String   // JSON string
  useCases    String   // JSON string
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model Inquiry {
  id          String   @id @default(cuid())
  refCode     String   @unique
  customerType String   // general | private | government
  name        String
  phone       String
  email       String?
  lineId      String?
  company     String?
  projectType String?
  province    String?
  delivery    String?
  budget      String?
  govRef      String?
  message     String?
```

```

    items      String    // JSON string – cart items
    attachments String?   // JSON string
    status      String    @default("new")
    createdAt   DateTime  @default(now())
    updatedAt   DateTime  @updatedAt
  }

  model Admin {
    id          String    @id @default(cuid())
    username     String    @unique
    passwordHash String
    createdAt   DateTime  @default(now())
  }

```

1.4 สร้าง .env

```

DATABASE_URL="file:./dev.db"
JWT_SECRET="change-this-to-a-random-32-char-string-in-prod"
RESEND_API_KEY="re_xxxxxxx"
ADMIN_EMAIL="admin@siammasterconcrete.com"
NEXT_PUBLIC_BASE_URL="http://localhost:3000"

```

1.5 สร้าง Prisma singleton — lib/prisma.ts

```

import { PrismaClient } from '@prisma/client'

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient }

export const prisma =
  globalForPrisma.prisma ?? new PrismaClient()

if (process.env.NODE_ENV !== 'production')
  globalForPrisma.prisma = prisma

```

1.6 สร้าง seed data — prisma/seed.ts

ดู spec ทั้งหมดได้ที่ <docs/seed-data.md>

```

import { PrismaClient } from '@prisma/client'
import bcrypt from 'bcryptjs'

const prisma = new PrismaClient()

async function main() {
  // Admin
  await prisma.admin.upsert({
    where: { username: 'admin' },

```

```

    update: {},
    create: {
      username: 'admin',
      passwordHash: await bcrypt.hash('Admin@SMC2024', 12),
    },
  })

// Products
await prisma.product.upsert({
  where: { slug: 'hollow-core-slab' },
  update: {},
  create: {
    slug: 'hollow-core-slab',
    nameTh: 'แผ่นพื้นสำเร็จรูป Hollow Core',
    nameEn: 'Hollow Core Slab',
    category: 'slab',
    descTh: 'แผ่นพื้นสำเร็จรูปที่มีช่องกลวงภายใน...',
    descEn: 'Hollow Core Slabs manufactured with European-standard Extruder...',
    specs: JSON.stringify({
      standard: 'มอก. 828-2531',
      thickness: ['15 ซม.', '20 ซม.', '25 ซม.', '30 ซม.'],
      width: '1.20 ม.',
      maxLength: '18 ม.',
      concreteGrade: 'fck = 40 MPa',
      unit: 'ตร.ม.',
    }),
    useCases: JSON.stringify([
      { th: 'คลังสินค้า / ศูนย์กระจายสินค้า', en: 'Warehouses / Distribution Centers' },
      { th: 'อาคารพาณิชย์', en: 'Commercial Buildings' },
    ]),
  },
})

// เพิ่ม Product 2 (Pile) และ Product 3 (Pole) ในลักษณะเดียวกัน
// ดูข้อมูลจาก docs/seed-data.md

console.log('Seed complete')
}

main().finally(() => prisma.$disconnect())

```

เพิ่มใน `package.json`:

```

"prisma": {
  "seed": "ts-node --compiler-options {\"module\": \"CommonJS\"} prisma/seed.ts"
}

```

```
npx prisma migrate dev --name init
npx prisma db seed
```

Task 2: i18n Setup

เวลา: ~1 ชั่วโมง

2.1 ตั้งค่า next-intl — `next.config.ts`

```
import createNextIntlPlugin from 'next-intl/plugin'

const withNextIntl = createNextIntlPlugin()

export default withNextIntl({})
```

2.2 สร้าง `middleware.ts`

```
import createMiddleware from 'next-intl/middleware'

export default createMiddleware({
  locales: ['th', 'en'],
  defaultLocale: 'th',
})

export const config = {
  matcher: ['/(?!api|_next|_vercel|.*\\.\\.*)'],
}
```

2.3 สร้าง `messages/th.json`

ดู copy ทั้งหมดที่ `docs/content.md`

```
{
  "nav": {
    "about": "เกี่ยวกับเรา",
    "products": "สินค้า",
    "portfolio": "ผลงาน",
    "contact": "ติดต่อ",
    "rfq": "ขอใบเสนอราคา"
  },
  "hero": {
    "eyebrow": "มาตรฐานอุตสาหกรรม · ก่อตั้งปี 2537",
    "title_line1": "คอนกรีตอัดแรง",
    "title_line2": "มาตรฐาน",
    "title_line3": "สูงสุด",
  }
}
```

```

    "subtitle": "ผู้ผลิตคอนกรีตอัดแรงสำเร็จรูป ผ่านมาตรฐาน มอก. ...",
    "cta_primary": "ขอใบเสนอราคา",
    "cta_secondary": "ดูสินค้าทั้งหมด"
  }
}

```

2.4 สร้าง `messages/en.json` — ใช้ key เดิม ภาษาอังกฤษ (ดูจาก `docs/content.md`)

2.5 `app/[locale]/layout.tsx`

```

import { NextIntlClientProvider } from 'next-intl'
import { getMessages } from 'next-intl/server'

export default async function LocaleLayout({
  children,
  params: { locale },
}: {
  children: React.ReactNode
  params: { locale: string }
}) {
  const messages = await getMessages()
  return (
    <html lang={locale}>
      <body>
        <NextIntlClientProvider messages={messages}>
          {children}
        </NextIntlClientProvider>
      </body>
    </html>
  )
}

```

Task 3: Navbar + Footer

เวลา: ~1.5 ชั่วโมง | ไฟล์: `components/layout/Navbar.tsx`, `Footer.tsx`

CSS Variables (เพิ่มใน `globals.css`):

```
:root {  
  --navy:    #1A1F2E;  
  --surface: #2C3347;  
  --border:  #3D4460;  
  --gold:    #C8922A;  
  --gold2:   #E8A838;  
  --chalk:   #F0EDE8;  
  --stone:   #8A8F9E;  
  --green:   #2A9D5C;  
}
```

Navbar.tsx — ดู mockup `v2-01-landing.html` สำหรับ layout:

- Logo: "S" mark (gold) + "Siam Master Concrete" + subtitle
- Links: เกี่ยวกับเรา | สินค้า | ผลงาน | ติดต่อ
- Right: [TH/EN toggle] [🌐 badge] [ขอใบเสนอราคา button]
- Sticky + backdrop-blur
- Language toggle: ใช้ `useRouter` + `usePathname` จาก next-intl

Task 4: Landing Page

เวลา: ~3 ชั่วโมง | ไฟล์: `app/[locale]/page.tsx` + components

4.1 HeroSection

- Layout: 2 คอลัมน์ (55% text | 45% product cards)
- Product cards: floating animation (CSS keyframes)
- Stats: 30 ปี | 3 มอก. | X โครงการ
- ดู mockup: `v2-01-landing.html` → `.hero`, `.hero-1`, `.hero-r`

4.2 TrustStrip

- Horizontal marquee-style
- 4 items: มอก. | 30 ปี | โครงการ | จัดส่งทั่วโลก

4.3 StrengthsSection

- 4 cards grid
- Icon + Title + Description
- ดู copy ที่ `docs/content.md` section 3.3

4.4 UseCaseSection

- 4 cards: Infrastructure | Building | Industrial | Utility
- ดู copy ที่ `docs/content.md` section 3.4

4.5 CTABand

- Full-width gold gradient background

- Title + Subtitle + Button
- ดู mockup: `.cta-band`

Task 5: Product Catalog

เวลา: ~4 ชั่วโมง

5.1 API Routes

`app/api/products/route.ts` — GET all products

```
import { prisma } from '@lib/prisma'
import { NextResponse } from 'next/server'

export async function GET(req: Request) {
  const { searchParams } = new URL(req.url)
  const category = searchParams.get('category')

  const products = await prisma.product.findMany({
    where: category ? { category } : undefined,
    orderBy: { createdAt: 'asc' },
  })

  return NextResponse.json(products.map(p => ({
    ...p,
    specs: JSON.parse(p.specs),
    useCases: JSON.parse(p.useCases),
  })))
}
```

`app/api/products/[slug]/route.ts` — GET single product

```
export async function GET(
  req: Request,
  { params }: { params: { slug: string } }
) {
  const product = await prisma.product.findUnique({
    where: { slug: params.slug },
  })
  if (!product) return NextResponse.json({ error: 'Not found' }, { status: 404 })
  return NextResponse.json({
    ...product,
    specs: JSON.parse(product.specs),
    useCases: JSON.parse(product.useCases),
  })
}
```

5.2 Product List Page

app/[locale]/products/page.tsx

- Fetch `/api/products` (server component)
- Layout: sidebar filter + product grid
- Filter by category (query param)
- ดู mockup: `v2-02-products.html`

5.3 ProductCard Component

```
// components/products/ProductCard.tsx
type Props = {
  product: Product
  locale: string
}
```

- Image | Badges (มอก.) | Category | Name | Specs chips
- Buttons: รายละเอียด | Datasheet | เพิ่มลงตะกร้า

5.4 Product Detail Page

app/[locale]/products/[slug]/page.tsx

- Full spec table
- Use cases list
- Download Datasheet button
- เพิ่มลงตะกร้า button (ใหญ่)

Task 6: RFQ Form + Email

เวลา: ~4 ชั่วโมง

6.1 ref-code generator — lib/ref-code.ts

```
import { prisma } from './prisma'
import { format } from 'date-fns'

export async function generateRefCode(): Promise<string> {
  const today = format(new Date(), 'yyyyMMdd')
  const prefix = `SMC-${today}-`

  const count = await prisma.inquiry.count({
    where: { refCode: { startsWith: prefix } },
  })

  return `${prefix}${String(count + 1).padStart(4, '0')}`
}
```

6.2 Email service — lib/email.ts

```
import { Resend } from 'resend'

const resend = new Resend(process.env.RESEND_API_KEY)

export async function sendAdminNotification(inquiry: {
  refCode: string
  name: string
  phone: string
  customerType: string
  items: string
}) {
  await resend.emails.send({
    from: 'noreply@siammasterconcrete.com',
    to: process.env.ADMIN_EMAIL!,
    subject: `[RFQ] ${inquiry.refCode} - ${inquiry.name}`,
    html: `
      <h2>คำขอใบเสนอราคาใหม่</h2>
      <p><strong>รหัส:</strong> ${inquiry.refCode}</p>
      <p><strong>ชื่อ:</strong> ${inquiry.name}</p>
      <p><strong>โทร:</strong> ${inquiry.phone}</p>
      <p><strong>ประเภท:</strong> ${inquiry.customerType}</p>
      <h3>รายการสินค้า:</h3>
      <pre>${inquiry.items}</pre>
    `,
  })
}

export async function sendCustomerConfirmation(inquiry: {
  refCode: string
  name: string
  email: string
}) {
  if (!inquiry.email) return
  await resend.emails.send({
    from: 'noreply@siammasterconcrete.com',
    to: inquiry.email,
    subject: `ยืนยันคำขอใบเสนอราคา - ${inquiry.refCode}`,
    html: `
      <h2>ขอบคุณสำหรับคำขอ ${inquiry.name}</h2>
      <p>รหัสอ้างอิงของคุณ: <strong>${inquiry.refCode}</strong></p>
      <p>ทีมงานจะติดต่อกลับภายใน 1 วันทำการ</p>
    `,
  })
}
```

6.3 RFQ API — app/api/rfq/route.ts

```

import { prisma } from '@lib/prisma'
import { generateRefCode } from '@lib/ref-code'
import { sendAdminNotification, sendCustomerConfirmation } from '@lib/email'
import { NextResponse } from 'next/server'

export async function POST(req: Request) {
  const body = await req.json()

  // Validate required fields
  if (!body.name || !body.phone || !body.customerType) {
    return NextResponse.json({ error: 'Missing required fields' }, { status: 400 })
  }

  const refCode = await generateRefCode()

  const inquiry = await prisma.inquiry.create({
    data: {
      refCode,
      customerType: body.customerType,
      name: body.name,
      phone: body.phone,
      email: body.email ?? null,
      lineId: body.lineId ?? null,
      company: body.company ?? null,
      projectType: body.projectType ?? null,
      province: body.province ?? null,
      delivery: body.delivery ?? null,
      budget: body.budget ?? null,
      govRef: body.govRef ?? null,
      message: body.message ?? null,
      items: JSON.stringify(body.items ?? []),
      status: 'new',
    },
  })

  // Send emails (non-blocking – don't await)
  sendAdminNotification({ ...inquiry, items: inquiry.items }).catch(console.error)
  sendCustomerConfirmation({ refCode, name: inquiry.name, email: inquiry.email ?? '' }).catch(console.error)

  return NextResponse.json({ refCode })
}

```

6.4 Track API — `app/api/rfq/[ref]/route.ts`

```

export async function GET(
  req: Request,
  { params }: { params: { ref: string } }
) {

```

```
const inquiry = await prisma.inquiry.findUnique({
  where: { refCode: params.ref },
  select: {
    refCode: true,
    status: true,
    customerType: true,
    name: true,
    createdAt: true,
    items: true,
  },
})
if (!inquiry) return NextResponse.json({ error: 'Not found' }, { status: 404 })
return NextResponse.json({ ...inquiry, items: JSON.parse(inquiry.items) })
}
```

6.5 RFQ Form Page

app/[locale]/rfq/page.tsx

- 3 แท็บ: หัวไป | เอกชน | ภาครัฐ (switch form fields)
- Client validation ก่อน submit
- POST ไป /api/rfq
- Redirect → /rfq/success?ref=SMC-...
- ดู mockup: v2-03-rfq.html
- Cart items panel ด้านขวา (อ่านจาก ?items=... query หรือ localStorage)

6.6 Success + Track pages

app/[locale]/rfq/success/page.tsx — แสดง refCode + ปุ่ม Track app/[locale]/rfq/track/page.tsx — input refCode → GET /api/rfq/[ref] → แสดง status

Task 7: About Page

เวลา: ~1 ชั่วโมง | app/[locale]/about/page.tsx

Sections:

- Hero: "30 ปีแห่งความเชี่ยวชาญ"
- Timeline/History paragraph
- Vision + Mission (2 column)
- Certifications: 3 มอก. cards
- ดู copy ทั้งหมดที่ docs/content.md section 4

Task 8: Deploy

เวลา: ~2 ชั่วโมง

8.1 Environment Variables บน Vercel

```
DATABASE_URL      → ใช้ Vercel Postgres (connection string)
JWT_SECRET        → random 32+ chars
RESEND_API_KEY    → จาก resend.com
ADMIN_EMAIL       → อีเมลรับแจ้งเตือน
NEXT_PUBLIC_BASE_URL → https://your-app.vercel.app
```

8.2 เปลี่ยน schema provider สำหรับ prod

ใช้ `DATABASE_URL` เป็น postgres connection string — Prisma detect อัตโนมัติ ถ้าใช้ `postgresql` provider

หรือ maintain 2 schema: `schema.prisma` (sqlite dev) + ตั้งค่า `DATABASE_URL` บน Vercel เป็น `postgresql://...`

8.3 Checklist ก่อน deploy

- ☐ ทดสอบ RFQ form end-to-end (submit → email ถึง admin)
- ☐ ทดสอบ track page ด้วย refCode จริง
- ☐ ทดสอบ mobile responsive ทุกหน้า
- ☐ ตรวจสอบ console errors ไม่มี
- ☐ ตรวจสอบ image paths (ถ้าไม่มีรูปจริง ใช้ placeholder ก่อน)

คำสั่งที่ใช้บ่อย

```
# Dev server
npm run dev

# Prisma
npx prisma studio          # เปิด DB GUI
npx prisma migrate dev     # สร้าง migration
npx prisma db seed         # ใส่ seed data
npx prisma generate        # regenerate client

# Build + check
npm run build
npm run lint
```

สิ่งที่ตั้งใจทำใน Phase 2 (ยังไม่ต้องทำตอนนี้)

- Cart System (localStorage + CartContext + CartPanel)
- Admin Dashboard (login + JWT + inquiry list + CRUD products)
- Portfolio page
- Excel export